Introduction to Programming in C

Course Code: **IDC 112**

Reji J

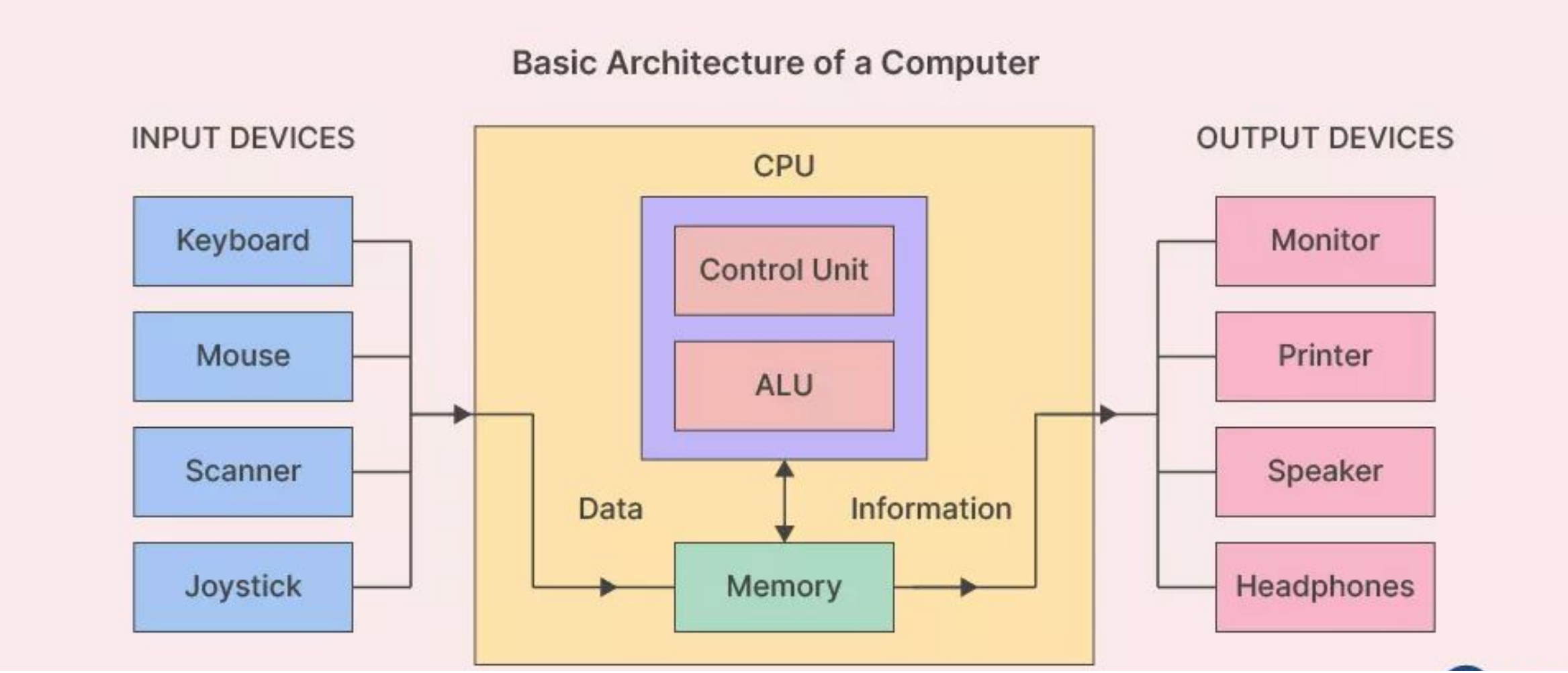
School of Data Science

IISER Thiruvananthapuram

Introduction to computer architecture & components.

Introduction to computer architecture and components.

- Computer architecture is the study of how computers are designed and organized.
- A computer is a general-purpose device that processes information to yield meaningful results (e.g., Desktop, Laptop, Tablet, Smartphone).
- A computer is made up of **various components** that work together and **communicate with each other.**
- The connection and interaction between these components is known as computer architecture.
- Computer architecture describes how hardware components are **interconnected** and how they **interact to perform tasks.**
- Understanding computer architecture is essential for understanding how computers operate, process data, and execute programs.


Basic Architecture of a Computer
INPUT DEVICES
Keyboard
Mouse
Scanner
Joystick
CPU
Control Unit
ALU
Data
Information
Memory
OUTPUT DEVICES
Monitor
Printer
Speaker
Headphones

Von Neumann Architecture

- Proposed by **John Von Neumann** in **1945**.
- Von Neumann architecture is based on the **stored-program concept**, where **instruction data** and **program data** share the same memory.
- This design is **still** widely **used** in most **modern computers**.
- It features a computer model where the **CPU**, **Memory**, and **Input / Output** devices share a single communication system.
- Both data and instructions are **stored** in the **same memory space.**
- The CPU operates sequentially, following a cycle of **fetching**, **decoding**, and **executing** instructions.
- A single bus system transfers data between the CPU, memory, and I/O devices.
- The design is simple and flexible, making it ideal for general-purpose computers.

How does Computer Works?

- The CPU is the heart of the computer, responsible for executing programs.
- Programs and data are stored in memory.
- Input devices (e.g., keyboard, mouse) interface with the CPU to provide commands.
- Output devices (e.g., speakers, monitor) receive processed data from the CPU.

Example: Playing a song on a computer

- The song (data) is stored in memory.
- An MP3 player (program) is also stored in memory.
- When you input a command to play the song, the CPU executes the MP3 player program.
- The CPU processes the data (song) and outputs the sound through speakers.

Example: Playing a video

- The video file (data) is stored in memory.
- A video player (program) is used to play the video.
- The CPU executes the video player program based on your input.
- The video is then displayed on the monitor or projector.

- The CPU executes programs, processes data, takes commands from input devices, and provides output through output devices.

Example

- Von Neumann Architecture is used in computer systems.
- First, you have to understand the difference between **programs** and **data**:
  a. **Data:** Information stored in memory (e.g., a song, a PPT file).
  b. **Program:** Software used to manipulate data (e.g., MP3 player, Microsoft PowerPoint).

Example: Editing a PPT file

- The **PPT file** (data) is stored in memory.
- **Microsoft PowerPoint** (program) is used to open and edit the file.
- The **CPU executes the program**, **reads the data**, and can **modify the data**.
- When it access the data, at that time it may **read** the data as well as it may **write** the data

- In Von Neumann Architecture, both data and programs are stored in the same memory.
- The CPU accesses this shared memory to read or write data while executing programs.
- This shared memory system, where both data and programs are stored together, defines the **Von Neumann Architecture**

Different Components of a Computer Architecture.

- Computer system has five basic units that help the computer to perform operations, which are given below:

1. **Input Unit**
2. **Output Unit**
3. **Memory Unit**
4. **Arithmetic Logic Unit**
5. **Control Unit**

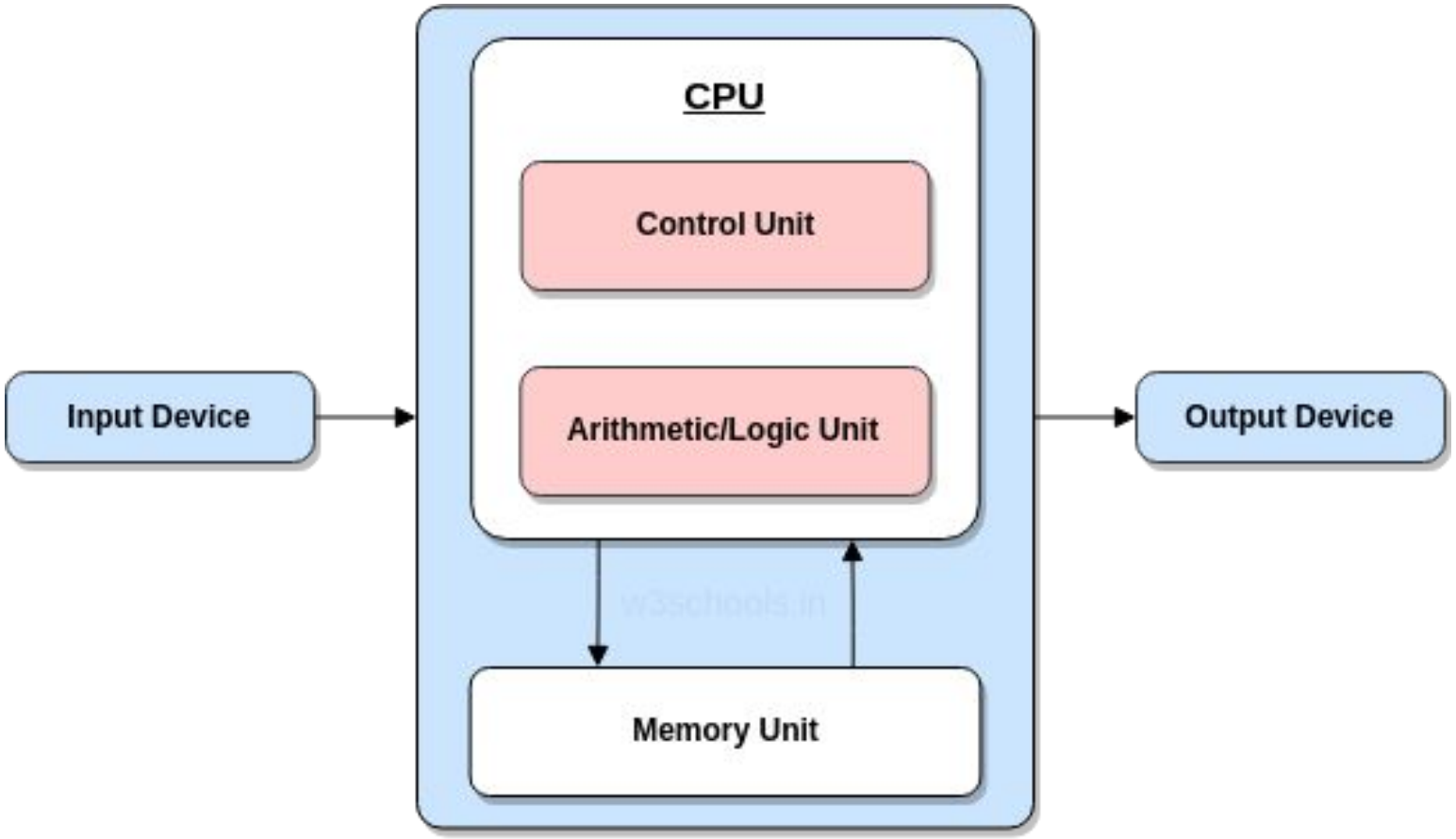


Fig: Von-Neumann Architecture

1. Input Unit

- A **computer** will **only respond** when a **command** is **given** to the **device**.
- These **commands** can be **given using** the input unit or the **input devices.**

For example:

Using a **keyboard** we can **type things** on a **Notepad** and the **computer processes** the entered **data** and then **displays** the **output** of the same of the screen.

- The **data** entered can be in the form of **numbers**, **alphabet**, **images**, etc.
- We enter the information using an input device, the processing units **convert** it into **computer understandable languages** and then the final **output** is received by a **human-understandable language.**

2. Output Unit

- When we command a computer to perform a task, it reverts for the action performed and gives us a **result**. This result is called **output**.
- There are various output devices connected to the computer. The most basic of which is a **monitor**. Whatever we write using a keyboard or click using a mouse, is all **displayed** on the **monitor**.
- Thus, the output unit gives us the **final result** once the entire processing is done within the mechanism of a device.

For example:

When we visit an ATM, we enter our details like language, pin, amount to be withdrawn, etc.

then the final money which the **cash dispenser** releases is our outcome.

In this case, the **cash dispenser** acts as an **output unit**.

3. Memory Unit

- When we enter the data into the computer using an input device, the entered information **immediately gets saved in the memory unit** of the Central Processing Unit (CPU).
- Because of the presence of some existing programming, the Memory Unit transmits the data further to the other parts of the CPU.
- Similarly, when the output of our command is processed by the computer, **it is saved in the memory unit** before giving the output to the user.
- The memory unit of a computer system can be **divided into two categories**:

  Primary Memory: This memory is used to store the data which is being **currently executed**. It is used for **temporary storage of data**. The data is lost, when the computer is switched off. **RAM** is used as primary storage memory.

  Secondary Memory: The secondary memory is **slower** and cheaper than primary memory. It is used for **permanent storage of data**. Commonly used secondary memory devices are **hard disk, pen drives** etc.

4. Control Unit

- This is the core unit which **manages** the **entire functioning** of the computer device.
- It is one of the most essential components of the computer system.
- The Control Unit **collects the data** entered using the input unit, **leads** it on for **processing** and once that is done, receives the output and presents it to the user. It can be said to the **centre of all processing actions** taking place inside a computer device.
- Basically, the **instructions taken**, **interpretation of entered data**, **issuing signals to execute the data** and then finally **retrieving the data** is all done in the Control Unit.
- It controls all other units of the computer. It controls the **flow of data** and **instructions to** and **from** the memory unit to ALU. Thus it is also known as **central nervous system** of the computer.

5. Arithmetic & Logic Unit (ALU)

- All the calculations are performed in ALU of the computer system. The ALU can perform basic operations such as addition, subtraction, division, multiplication etc.
- Whenever calculations are required, the **Control Unit** transfers the data from **Memory Unit** to **ALU**. When the operations are done, the result is transferred back to the **Memory Unit.**
- As the name suggests, all the **mathematical calculations** or **arithmetic operations** are performed in the Arithmetic and Logical Unit of the CPU.
- It can also perform actions like a **comparison of data** and **decision-making actions.** The ALU comprises circuits using which **addition**, **subtraction**, **multiplication**, **division** and other numerical based calculations can be performed.

Central Processing Unit (CPU)

- The Central Processing Unit is the core of any computer devices.
- It comprises three major components of the computer which have been discussed above:
    - **Memory Unit**
    - **Control Unit**
    - **Arithmetic and Logical Unit**
- All these three units are elements of CPU and together help in the efficient working and processing of data.
- It is also known as the "**Brain of Computer**" and no action can be conducted by a device without the execution and permission of the Central Processing Unit.
- The device is a close-knit circuit comparison microprocessor which helps in fetching the data and providing suitable results to the user. Thus, **CPU is the main processing unit of the computer.**

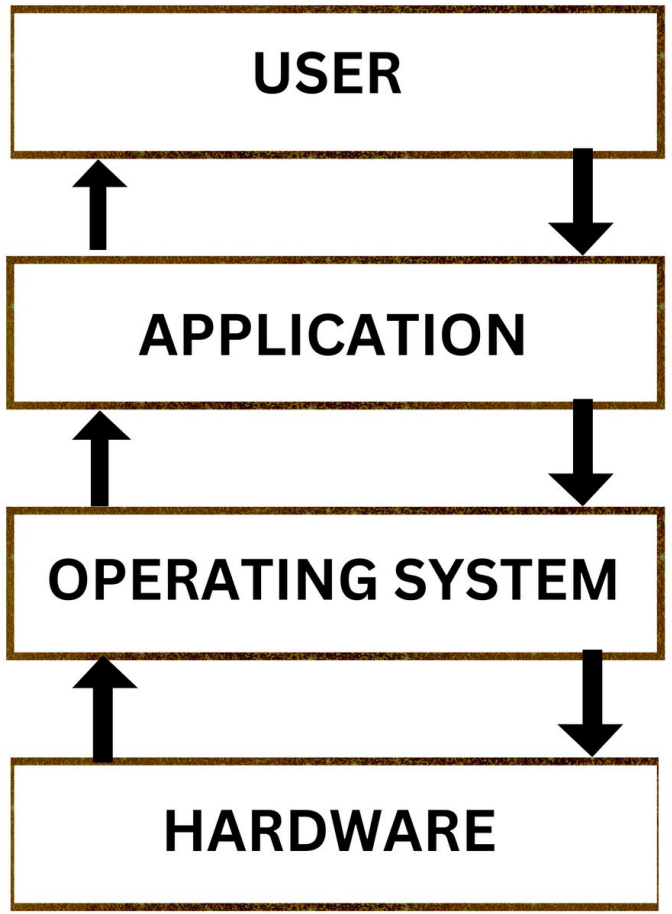
Introduction Linux OS &
Linux Command Line Interface

Introduction to Linux OS

What is an **Operating System?**

- An operating system is a set of software layers between you and your computer's hardware.
- The operating system (or OS) is a piece of software that controls all other application programs and helps you manage the hardware and software of your computer.
- An operating system that directly manages a system’s hardware and resources, like CPU, memory, and storage.
- **Examples** of popular operating systems are

  Windows, Linux, MacOS, and Android.



Introduction to Linux OS

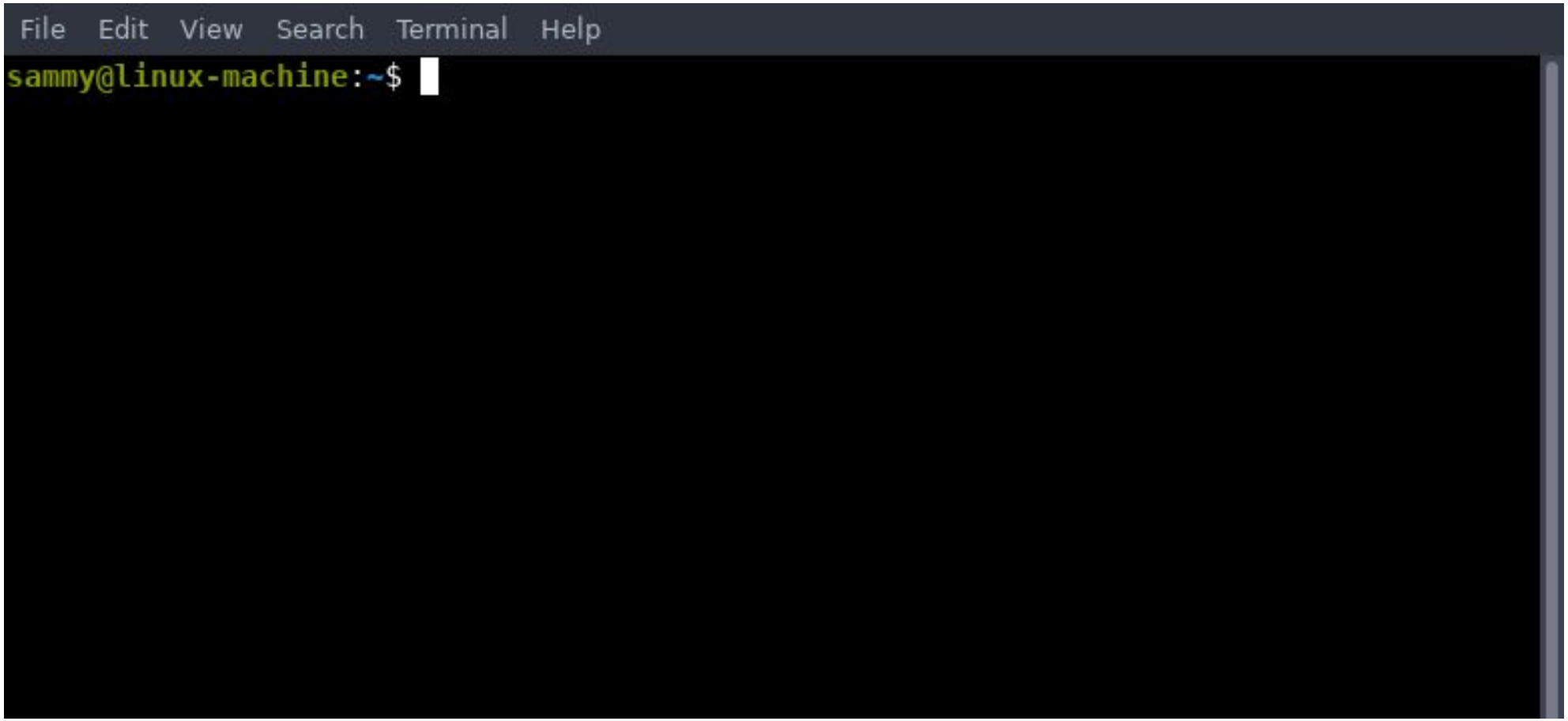
- Linux is an **open-source** operating system (OS) and completely free for all users.
- Unlike commercial OS like Windows and Mac, Linux is **free** and **open-source**, requiring no license fees.
- Under licenses like **GNU (General Public License)**, anyone can **access**, **use**, **modify**, and **distribute** the Linux source code for any purpose.
- Linux is one of the most widely used **UNIX**-based operating systems.
- Linux-based operating systems are known as **Linux distributions** or **distros** (e.g., Debian, Ubuntu, Fedora, CentOS).
- Linux has a different **filesystem structure** compared to Windows or MacOS
- Linux relies heavily on the **command line interface**, unlike most personal computers that use graphical interfaces.

History of Linux

- Linux is an **operating system** or a **kernel** which germinated as an idea in the mind of young and bright **Linus Torvalds** when he was a computer science student.
- He used to work on the **UNIX OS (proprietary software)** and thought that it needed improvements.
- However, when his suggestions were rejected by the designers of UNIX, he thought of launching an OS which will be **receptive to changes, modifications suggested by its users**.
- **So, Linus devised a Kernel** named Linux in **1991**.
- Though he would need programs like **F**ile **M**anager, **D**ocument **E**ditors, **A**udio -**V**ideo **p**rograms to run on it.
- As time passed by, he collaborated with other **programmers in places like MIT** and applications for Linux started to appear. So around **1991**, a working Linux operating system with some applications was **officially launched**, and this was the start of one of the **most loved and open-source OS options available today**.
- The earlier versions of Linux OS were not so **user-friendly** as they were in use by computer programmers and **Linus Torvalds never had it in mind to commercialize** his product. This definitely curbed the Linux's popularity as other commercially oriented Operating System **Windows** got famous.

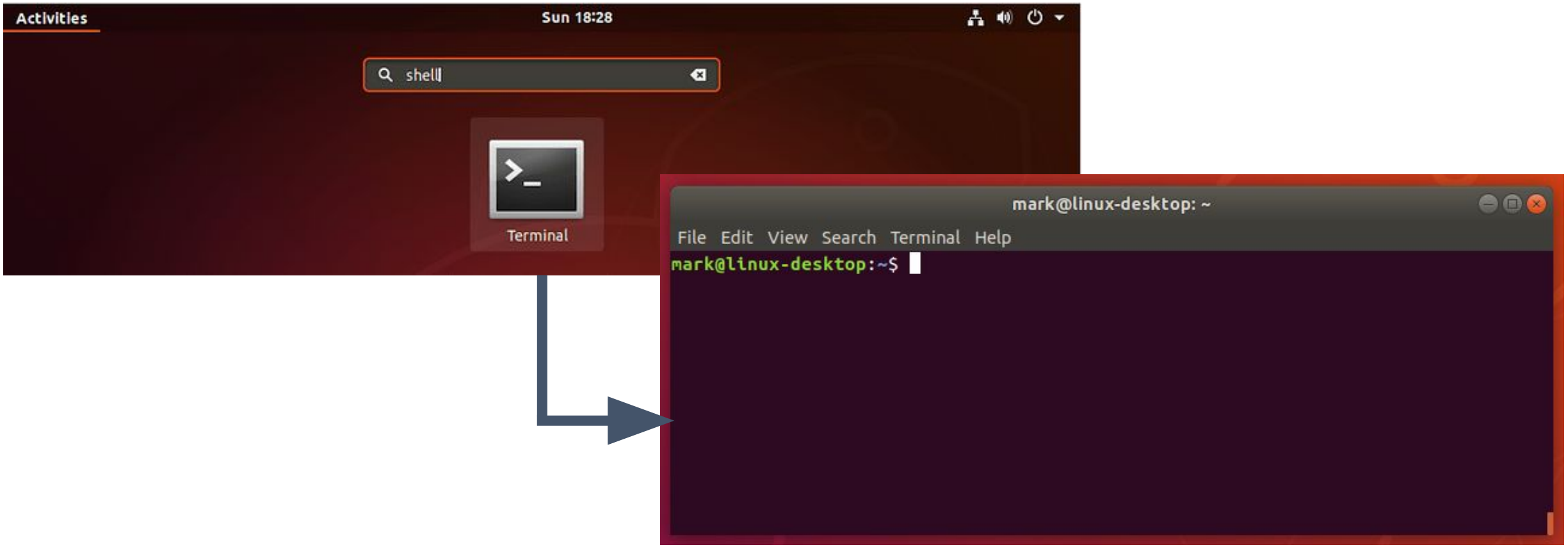
Terminal

- A ***terminal*** is an input and output environment that presents a text-only window running a shell.



How do you open the command line interface (CLI) in Linux?

- Linux has a simple way of accessing the CLI: use the shortcut **CTRL+ALT+T.**
- OR you can find a launcher for the terminal by clicking on the Activities item at the top left of the screen, then typing the first few letters of “terminal”, “command”, “prompt” or “shell”.



Basic Linux Commands to Run in the Terminal

- A command line interface (CLI) has many different commands for different types of tasks. The commands vary slightly between operating systems. We give several different command examples next.

File system commands

CLI command	Windows	Linux
Change directory	**cd**	**cd**
List files	**dir**	**ls**
Create directory	**mkdir**	**mkdir**
Remove directory	**rmdir**	**rmdir**
Copy files	**copy**	**cp**
Move files	**move**	**mv**
Rename files	**ren**	**mv**
Display files	**type**	**cat**
Delete files	**del**	**rm**

Basic Linux Commands to Run in the Terminal

- Becoming a **Linux expert** requires you to be **comfortable with using a terminal.**
- Any administrative task, including file **manipulation**, **package installation**, and **user management**, can be accomplished through the **terminal**.
- The terminal is interactive: you specify commands to run and the terminal outputs the results of those commands. To execute any command, you type it into the prompt and press ENTER.

  Example of the default Ubuntu 22.04 command prompt:

  ```
  user@lhcpc:~$
  ```

  Here is a breakdown of the composition of the command prompt:

  - **user**: The *username* of the current user
  - **lhcpc**: The *hostname* of the server
  - **~**: (**The *current directory)*** : path of the current user's *home directory*; ie, it represents **/home/iisertvm**
  - **$**: (**The prompt symbol).** This denotes the end of the command prompt, after which the user's keyboard input will appear

Basic Linux Commands to Run in the Terminal

- **The clear command**

  Clears all previous commands that were run in the current terminal.

  This clears the screen from previous commands in the terminal.

  ```
  user@iisertvm:$ clear
  ```

- **How to open files**

  The open command lets you open a file or directory in the graphical user interface (GUI) outside the terminal.

  xdg-open filename or directory name

Example:

```
user@iisertvm:$ xdg-open acad_calender.pdf
```

Work with Directories in Linux

- Directories are like folders, and you can create, delete, and perform all functions on them through your system interface with a mouse or cursor.

- The following commands let you perform different operations on **directories**:

  1) **pwd** (present working directory)

  2) **cd** (change directory)

  3) **ls** (list)

  4) **mkdir** (make directory)

  5) **rmdir** (remove directory)

Contd…

- **The pwd command**

  Whenever you feel lost in the filesystem, call the pwd command to know where you are.

  It takes no argument.

  ```
  user@iisertvm:$ pwd
  ```

  It should print the current folder/directory path where you currently are.

- **The cd <path> command**

  You can change your **current directory** with the cd command (Change Directory).

  Just like going back and forth between folders when using the GUI.

  ```
  user@iisertvm:$ cd Documents/articles
  ```

  This command takes me to a folder called **articles** inside my **Documents** folder.

Contd...

Let's see what else you can do with cd

cd ~

- The cd is also a shortcut to get back into your **home directory.**
- Typing **cd ~** has the same effect.

```
user@iisertvm:$ cd ~
```

cd . .

- To go to the **parent directory** (the one just above your current directory in the directory tree), type `cd ..`

```
user@iisertvm:$ cd ..
```

Contd…

- **The ls command**

  Inside a folder you can **list all the files** that the folder contains using the **ls** command.

  It takes no arguments.

  ```
  user@iisertvm:$ ls
  ```

- **The mkdir <directoryName> command**

  we can create your own directories/folders with **mkdir** command. which stands for "**make directory**"

  ```
  user@iisertvm:$ mkdir myFolder
  ```

- **The rmdir <directoryName> command**

  When a directory is empty, you can use **rmdir** to remove or delete the directory.

  which stands for "**remove directory**"

  ```
  user@iisertvm:$ rmdir myFolder
  ```

How to Work with Files in Linux

- In this section, you'll learn how to recognize, create, remove, copy, and move files using the following commands:

  1) **touch** (creates an empty file)

  2) **rm** (remove file)

  3) **cp** (copy)

  4) **mv** (move)

Contd…

- **The touch <filename> command**

  One easy way to create an empty file is with **touch** like this:

  ```
  user@iisertvm:$ touch file1.txt
  ```

- **The rm <filename> command**

  The rm command is used to delete files.

  When you no longer need a file, use **rm** to remove it.

- **The cp command**

  The **cp** command is equivalent to the **copy-paste** command in Windows.

  To copy a file, use **cp** with a file name and a new file name argument.

  ```
  user@iisertvm:$ cp <source> <destination>
  ```

  ```
  user@iisertvm:$ cp   hello.txt   hellocopy.txt
  ```

  ```
  root@ubuntu:~# ls
  Sample
  root@ubuntu:~# cp Sample Sample-Copy
  root@ubuntu:~# ls
  Sample  Sample-Copy
  root@ubuntu:~#
  ```

Contd…

- **The mv command**

  The **mv** commands is equivalent to the **cut-paste** command in Windows.

  But since Linux doesn't really have a command for renaming files,

  we also use the **mv** command to **rename** files and folders.

```
user@iisertvm:$ mv <source> <destination>
```

```
user@iisertvm:$ mv   hello.txt   hellocopy.txt
```

```
root@ubuntu:~# ls
Sample
root@ubuntu:~# mv Sample Sample-Copy
root@ubuntu:~# ls
Sample-Copy
root@ubuntu:~#
```

In the above case, since we were moving the file within the same directory, it acted as a rename.

The file name is now changed.

- Use this option to copy a file to another directory (destination).

```
user@iisertvm:$ mkdir myfolder
```

```
user@iisertvm:$ mv   hello.txt   myfolder
```

Activity – Linux Command Line

1. **Navigate to Desktop directory from the current working directory.**
2. **Make a directory named IMS24XXX. (XXX - last 3 digits of your roll number)**
3. **Check if the folder IMS24XXX is created by listing all files.**
4. **Go inside the IMS24XXX folder.**
5. **Create an empty file named hello.txt.**
6. **Open the hello.txt file, type Programming is fun inside, save it, and then close the file.**
7. **Verify your present working directory.**
8. **Create another empty file named hai.txt.**
9. **List all the files inside your IMS24XXX folder.**

Activity – Linux Command Line

10. **Make another directory named Backup.**
11. **Move hai.txt into the Backup folder.**
12. **Create a copy of hello.txt and name it data.txt.**
13. **Rename hello.txt to MyFile.txt.**
14. **Remove the Backup directory (make sure it's empty).**
15. **Delete the data.txt file.**
16. **Clear all commands from the Terminal.**
17. **Go back to your parent directory or home directory.**

Task

1. **Navigate to Desktop directory**

```
user@iisertvm:$ cd Desktop
```

2. **Type pwd to print the present working directory**

```
user@iisertvm:$ pwd
```

Task – CLI

3. Create a Directory

- Make a directory named **IMS24XXX**.

```
user@iisertvm:$ mkdir IMS24XXX
```

4. List Files

- Check if the folder **IMS24XXX** is created by listing all files.

```
ls
```

5. Navigate to the Directory

- Go inside the **IMS24XXX** folder.

```
cd IMS24XXX
```

6. Create a File

- Create an empty file named hello.txt.

```
touch hello.txt
```

7. Open the File

```
xdg-open hello.txt
```

- Open the **hello.txt** file.
- Type **Programming is fun** inside the file, save it, and then close the file.

Task – CLI

6. **Check Current Directory**
   - Verify your present working directory.

   `pwd`

7. **Create another File**
   - Create another empty file named hai.txt.

   `touch hai.txt`

8. **List Files Again**
   - List all the files inside your **IMS24XXX** folder.

   `ls`

9. **Create another Directory**
   - Make another directory named Backup.

   `mkdir Backup`

10. **Move a File**
    - Move hai.txt into the Backup folder.

    `mv hai.txt Backup`

11. **Copy a File**
    - Create a copy of hello.txt and name it data.txt.

    `cp hai.txt data.txt`

Task – CLI

12. **Rename a File**
    - Rename hello.txt to MyFile.txt.

```
mv hello.txt MyFile.txt
```

13. **Remove a Directory**
    - Remove the Backup directory.

```
rmdir Backup
```

14. **Remove a File**
    - Delete the data.txt file.

```
rm data.txt
```

15. **Clear the Terminal**
    - Clear all commands from the Terminal.

```
clear
```

16. **Return to Parent Directory**
    - Go back to your parent directory or home directory.

```
cd ..
```